Student Project — Backtesting the ProntoNLP Earnings-Call ATC Signal

Course: LLM-Driven Quant Research **Dataset:** `Earnings_ATC_until_2026-04-21.csv.zip` (≈422 MB compressed, ≈4.5 GB uncompressed) **Due deliverables:** code repository + backtest charts + research PDF **Hard rule: NO look-ahead bias.** A submission with any look-ahead failure does not pass, regardless of measured alpha.

---

1. The Dataset

The file is a single CSV: `Earnings_ATC_until_2026-04-21.csv` (≈4.47 GB uncompressed, ≈2.74 M rows, 609 columns). Each row is one **(earnings call, signal-aggregation slice)** record produced by ProntoNLP's NLP pipeline over S&P Global earnings-call transcripts.

1.1 Coverage

Date range	2010-01-04 → 2026-04-21
Total rows	2,740,437
Unique tickers	19,819
Document type	Earnings Calls (≈100%)
Sectors	All 11 GICS sectors (Industrials, Financials, Health Care, IT, Cons. Disc., Materials, Energy, Comm. Services, Real Estate, Cons. Staples, Utilities)
Countries	100 (US ≈55% — Canada, India, UK, Australia, Sweden, China, Japan, Germany, Brazil, France next)

1.2 Row structure: 9 SignalType slices per call

For each call the dataset contains up to **9 rows**, one per `SignalType`. The same NLP features are computed on different subsets of the transcript:

SignalType	Rows	What it covers
`Total`	376,790	Entire transcript
`Presentation`	373,808	Prepared remarks only
`Question`	351,494	Analyst questions only
`Answer`	359,535	Management answers only
`Executives`	376,036	All executives speaking
`CEO`	303,854	CEO sentences only
`CFO`	253,155	CFO sentences only
`Analysts`	343,534	Analyst sentences only
`delete`	2,231	**Drop these rows** (corrupt/invalidated)

Pick `Total` as your default; treat the speaker-specific slices as alternative signals to compare.

1.3 Identifier columns (first ~28 cols)

- `SignalType` — the slice (above)
- `DocDate` — call date (`YYYY-MM-DD`)
- `MOSTIMPORTANTDATEUTC` — datetime of the call. Parse the **hour** to determine BMO vs AMC (≥16 UTC → after-market-close; you must enter the trade the *next* trading day, not the same day).
- `Ticker` / `BESTTICKER` — exchange ticker. `BESTTICKER` is the cleanest field to use as your join key against an external price source (e.g., yfinance). See §6 for guidance on sourcing prices.
- `GVKEY` — Compustat key. Useful only if you have access to a price source keyed by gvkey (most students will not — use `BESTTICKER` instead).
- `COMPANYID` / `CIK` / `ISIN` / `KEYDEVID` / `DocID` — other IDs
- `COMPANYNAME` , `SIMPLEINDUSTRYDESCRIPTION` , `SECTOR` , `COUNTRY` , `EXCHANGE` , `EX_NAME` , `EX_CODE` , `EX_MIC` — descriptive
- `QTR_YEAR` — fiscal quarter
- `DOCSECTIONCOUNT` , `DOCSENTENCECOUNT` — transcript size
- `INGESTDATEUTC` — when ProntoNLP ingested the doc (informational; do not use as a feature)
- `Sentences` — count of scored sentences in this slice
- `HEADLINE` , `CORPUS` , `DOCTYPE` — bookkeeping

1.4 Signal columns (~580 numeric features)

(a) EventScore family — 12 columns, 4 score variants × {Pos, Neg, Score}: - `EventPos_1_1_1` , `EventNeg_1_1_1` , `EventsScore_1_1_1` - `EventPos_4_2_1` , `EventNeg_4_2_1` , `EventsScore_4_2_1` - `EventPos_3_1_0` , `EventNeg_3_1_0` , `EventsScore_3_1_0` - `EventPos_1_1_0` , `EventNeg_1_1_0` , `EventsScore_1_1_0`

The numeric suffixes encode internal ProntoNLP classifier configurations (window/threshold/normalization). `4_2_1` is the variant the production trading desk has historically used.

(b) `ATCClassifierScore` — single aggregated classifier output (the headline ATC score).

(c) AspectTheme matrix — ~567 columns, one per (Aspect × Theme × Magnitude × Sentiment):

```
AspectTheme_{Aspect}_{Theme} - {Magnitude} - {Sentiment}
```

- **Aspect (7):** `CurrentState`, `Forecast`, `Surprise`, `StrategicPosition`, `Fluff`, `Filler`, `Other` — the **temporal / informational** classification of the sentence
- **Theme (9):** `FinancialPerformance`, `OperationalPerformance`, `MarketAndCompetitivePosition`, `StrategicInitiatives`, `CapitalAllocation`, `RegulatoryAndLegalIssues`, `ESG`, `MacroeconomicFactors`, `Other` — the **business-domain** classification
- **Magnitude (3):** `High`, `Medium`, `Low`
- **Sentiment (3):** `Positive`, `Neutral`, `Negative`

Each cell is a count of sentences in the slice that fall into that bucket. See §1.6 for the formal definitions of every Aspect and Theme. **Drop the `Fluff` and `Filler` aspects** in your engineered features — they are textual-noise classes by design.

1.6 Aspects and Themes — formal definitions

The two categorical dimensions encoded in the `AspectTheme_*` column names are not arbitrary buckets — they come from the ProntoNLP transcript classifier and have specific definitions. The text below is taken verbatim from the official ProntoNLP overview deck (slides 6 and 7).

Aspects (7) — temporal / informational taxonomy

> *Aspects provide a temporal and informational taxonomy for categorizing events extracted from earnings calls. This classification helps distinguish between backward-looking performance commentary, forward-looking guidance, and strategic positioning statements.*

Aspect	Definition
Current State	Events focusing on recent performance or current operations, providing a snapshot of the company's present status and historical results.
Forecast	Forward-looking predictions about future performance, offering insights into management expectations and projections for the company's trajectory.
Surprise	New, unexpected external developments such as sudden windfalls or crises that deviate from normal expectations and can have immediate market impacts.
Strategic Position	Company strategic advantages and disadvantages, including new product development, technological innovations, competitive factors, market share dynamics, and intellectual property.
Fluff	Language lacking substantive content, including exaggerated or superlative statements without factual support. These do not provide meaningful information and are often overly promotional.
Filler	Boilerplate corporate language such as legal disclaimers, generic descriptions, routine acknowledgments, and technical procedural content from earnings calls.
Other	Events that do not fit any other aspect category, including cases where timing or context prevents proper classification.

> **Practical note for the backtest:** `Fluff` and `Filler` are noise classes by design — they capture sentences with no actionable information. They are useful as a *control* (a signal built only on Fluff/Filler should generate ≈0 alpha; if it doesn't, you have a bug). Do not feed them as predictive features into your main model.

Themes (9) — business-domain taxonomy

> *Thematic classification organizes earnings call content by business domain, enabling targeted analysis of different aspects of corporate performance. These nine themes capture the full spectrum of topics discussed during earnings calls.*

The nine themes are: **Operational Performance**, **Market and Competitive Position**, **Financial Performance**, **Strategic Initiatives**, **Capital Allocation**, **Regulatory and Legal Issues**, **ESG**, **Macroeconomic Factors**, **Other**.

> **Signal-construction note (from the source deck):** "*The loss function for the signal is (average of stock's price during 14 days after the earnings call) minus (average of stock's price during 14 days before the earnings call).*" — i.e. the underlying classifier was trained against a 14-day window straddling the call. Useful context when interpreting why certain horizons (1, 3, 5d) may show weaker signal than others (10, 20d).

Why the cross-product matters

A single sentence is tagged with **one Aspect × one Theme × one Magnitude × one Sentiment** — so the same Theme can show up under different Aspects with very different trading implications. Examples:

- `CurrentState_FinancialPerformance - High - Negative` → "Q3 revenue fell 18% YoY" → bearish, already-realized
- `Forecast_FinancialPerformance - High - Negative` → "We expect Q4 revenue to decline mid-teens" → bearish, forward-looking (often has bigger price impact)
- `Surprise_RegulatoryAndLegalIssues - High - Negative` → "We received an unexpected SEC inquiry" → tail-risk event
- `StrategicPosition_StrategicInitiatives - Medium - Positive` → "Our new product is gaining share" → competitive moat update

Aggregating naively across Aspects (e.g., summing all `*_FinancialPerformance` cells) destroys this structure. Best practice: keep the Aspect × Theme cross-product and let the model learn which combinations matter, or engineer interpretable per-Aspect and per-Theme features separately.

1.7 Consensus / KPI features — embedded in `ATCClassifierScore`

The ProntoNLP signal evolved across four versions (per the source deck, slide 3):

Version	Features
V1	FIEF Patterns + a fine-tuned RoBERTa sentiment model trained on 30,000 annotated sentences
V2 — AlphaLLM	LLMTag (2.7M tags), Polarity (3), Importance (3), Explanation, LLM Event (110 event types)
V3 — Aspects	Polarity (3), Importance (3), Aspects (7)
V4 — Aspects / Themes / Consensus Data	Polarity (3), Importance (3), Aspects (7), Themes (9), **Consensus (6 KPIs)**

The **Consensus** block in V4 captures, for each call, how the company's actual reported figures compare to the analyst-consensus estimate going into the call across **6 key financial KPIs:**

#	KPI	What it captures
1	**EBITDA**	Operating profitability surprise (earnings before interest, tax, depreciation, amortization)
2	**EPS (GAAP)**	The headline beat/miss number — earnings per share under GAAP
3	**Net Income (GAAP)**	Bottom-line profit surprise under GAAP
4	**Revenue**	Top-line surprise — the "did they grow?" number
5	**Capital Expenditure (CapEx)**	Investment-intensity surprise — signal of growth posture or retrenchment
6	**Free Cash Flow (FCF)**	Cash-generation surprise — often the cleanest profitability signal

For each KPI the classifier sees the company's reported value relative to consensus going into the call; this lets the model condition its language-derived signal on the surprise dimension (e.g., "*negative tone with a beat*" vs. "*negative tone with a miss*" are very different setups).

Important: the 6 KPIs are not exposed as separate columns in this dataset

A scan of all 609 columns confirms there are no `EBITDA` / `EPS` / `Revenue` / `CapEx` / `FCF` /consensus columns in the file. Instead, the 6 KPIs were used as **training inputs to the classifier** that produces the `ATCClassifierScore` column (see §1.8). In other words: when you read `ATCClassifierScore`, you are already implicitly receiving the consensus-aware view — the classifier has internalized the language signal *and* the KPI surprises into a single number.

For this project you should: - **Use `ATCClassifierScore` as your primary signal** (see §1.8) — it already encodes the consensus dimension. - **Do not attempt to rebuild the consensus features** by joining external data (e.g., I/B/E/S, FactSet, Refinitiv estimates). The classifier score already does this for you, and joining external estimate data is a notorious source of look-ahead bias (estimate revisions, vendor restatements, point-in-time data hygiene, etc.). - **If you want to add an explicit "surprise" feature on top of `ATCClassifierScore`**, derive it from price action *before* the call (T-1 vs. T-21 return, idiosyncratic alpha, sector-relative momentum) — these are pre-event and clean. - Treat the ATC dataset as self-contained and focus the project on extracting the most alpha possible from what's already in the file.

1.8 The headline classifier score: `ATCClassifierScore`

Of all 580+ feature columns, `ATCClassifierScore` is the single most important one to know. It is the **production-grade aggregated classifier output** — ProntoNLP's own model has already done the hard work of mapping the 567-cell AspectTheme matrix, the four EventScore variants, **and the 6 consensus KPI surprises** (EBITDA, EPS-GAAP, Net Income-GAAP, Revenue, CapEx, FCF — see §1.7) into a single number per call.

Recommended primary use: long/short ranking

On a given trading day (or week, or month), use `ATCClassifierScore` to **rank** all companies that reported earnings in the lookback window:

- **Long** the top quintile / decile / top-N by score
- **Short** the bottom quintile / decile / bottom-N
- Hold for the chosen horizon (1, 3, 5, 10, or 20 trading days)

This single-feature long/short construction is the **first model every student should build** — it serves as both a sanity check and an honest baseline. If your fancy multi-feature ML model doesn't beat the `ATCClassifierScore` quintile spread on a Sharpe basis, the ML model is not adding value.

Why `ATCClassifierScore` is a good primary signal

1. It's already calibrated against forward returns (the classifier was trained against the 14-day pre/post-call window mentioned in slide 7).
2. It's a **single number per (call, SignalType)** — no feature-engineering required, no risk of accidentally introducing look-ahead via feature aggregation choices.
3. It's available for the entire history (2010 → present) with consistent semantics.
4. Industry desks at S&P Global use it as the production trading score — replicating its quintile P&L is a meaningful first deliverable.

Required experiment using `ATCClassifierScore`

- Compute the **decile spread** (top decile minus bottom decile cumulative return) per universe and per horizon.
- Plot the cumulative L/S equity curve, drawdown, and rolling Sharpe.
- Report the IC of `ATCClassifierScore` vs. each forward-return horizon.
- Compare across `SignalType` slices (Total vs. CEO vs. CFO vs. Analysts) — the deck and prior research suggest the speaker-specific slices can produce differentiated signals.

Going beyond the headline score

After establishing the `ATCClassifierScore` baseline, you have two richer feature paths to explore:

1. **The base sparse matrix** — the ~567 raw `AspectTheme_*` cells. This is high-dimensional (most cells are 0 for any single call), but a regularized model (LightGBM, Lasso, etc.) can find non-linear interactions the headline score smooths over. Best for tree-based models that handle sparsity natively.
2. **Aggregated / engineered features** — per-Aspect totals, per-Theme totals, sentiment-weighted sums, magnitude-weighted scores, QoQ deltas, multi-quarter trends, sector-relative percentile ranks (computed point-in-time — see §3). These are usually the most informative inputs for linear and ridge-style models.

A typical strong-result project structure is: - **Baseline:** `ATCClassifierScore` decile L/S, no model - **Enhanced:** LightGBM trained on engineered features (50–150 cols) - **Stretch:** LightGBM on engineered features + selective sparse-matrix interactions

Always report all three so the grader can see the marginal value of complexity.

---

2. The Assignment

2.1 Goal

Build a **rigorous, look-ahead-free backtest** of the ATC signal on three universes — **S&P 500, S&P 1500, Russell 3000** — and produce an opinion on **how to use the signal in production** at one of three rebalance cadences: - **Daily** (event-driven): trade individual earnings prints as they come in - **Weekly**: rebuild a portfolio every Monday from events in the past N days - **Monthly**: rebuild on the first trading day of the month

You may pick the cadence you think is best, but you must justify the choice with backtest evidence (turnover, capacity, alpha decay).

2.2 Required experiments

For each universe (SP500, SP1500, RU3K): 1. **Single-feature IC analysis.** Compute Spearman IC of `ATCClassifierScore`, each `EventsScore_*`, and a small handful of engineered features against forward returns at horizons {1, 3, 5, 10, 20} days. Report by year and by sector. 2. **Quintile / decile portfolios.** Sort events into quintiles by signal at T-0, hold for the chosen horizon, report long-only, short-only, and long-short cumulative returns and Sharpe. 3. **Predictive model (walk-forward).** Train a model (your

choice — Ridge/LightGBM/XGBoost are fine) on engineered features. Use **expanding-window walk-forward** with one quarter as the test step. Train end ≤ 2019 to start, walk forward through 2026Q2. 4. **Rebalanced portfolio simulation.** Implement your chosen rebalance cadence. Track turnover, gross/net exposure, and post-cost Sharpe (assume **5 bps one-way** transaction cost). 5. **Robustness checks.** Sub-period (pre-2020 / 2020-2022 / 2023-2026), sector neutralization, market-cap bucket (mega/large/mid/small), parameter sensitivity.

2.3 Deliverables

1. **Code** — clean, reproducible, in a single repo. A `README.md` with one command to reproduce all results from the raw CSV.
2. **Backtest charts** — PDF or PNG bundle. At minimum: cumulative L/S equity curve per universe, IC heatmap (feature × horizon), quintile spread chart, drawdown chart, turnover bar chart.
3. **Research PDF** — full write-up. Sections: data description, methodology, look-ahead audit, results per universe, recommended deployment (cadence + position-sizing rule), risks/limitations, future work. Target 15–25 pages.
4. **Look-ahead bias audit checklist** — a one-page document signing off that each of the items in §3 was checked.

---

3. Look-Ahead Bias — The Audit You Must Pass

Look-ahead bias is the single most common way student backtests look brilliant and then lose money in production. The grader will run automated checks for these. **Self-audit before submitting:**

1. **Entry timing — AMC vs BMO.** - Parse `MOSTIMPORTANTDATEUTC`. If the **hour ≥ 16 UTC**, the call is after-market-close → entry is the **next** trading day's close, not the call day's close. - If the call is before-market-open (hour < 13 UTC, roughly), entry can be the same trading day's close. - Anything in between needs a documented rule. Pick one and stick to it.
2. **Forward returns are targets, never inputs.** - `Return_1d`, `Return_3d`, etc. are computed *from* the price data after entry. Never feed them — or any function of them — back into model features. (Even a "filter shorts where 1-day return is positive" is look-ahead.)
3. **Cross-sectional features must be point-in-time.** - Z-scores, percentile ranks, sector means etc. computed across "all events in the same quarter" leak the future of that quarter. Use only events that have already happened by the entry date (a.k.a. an **expanding-window** percentile).
4. **Feature selection is part of training.** - Spearman / mutual-info ranking, PCA fits, target-encoding etc. must be done on **training fold only**. Refit at every walk-forward step.
5. **Imputation and scaling are part of training.** - `StandardScaler.fit` and median imputation must use training data only. `fit_transform(train); transform(test)`.
6. **Universe membership is point-in-time.** - Today's S&P 500 is not 2014's S&P 500. Either use a historical constituents file or accept the survivorship-bias caveat in writing.
7. **`INGESTDATEUTC` ≠ availability date.** - Some calls were ingested by ProntoNLP days after the actual call. For the strictest backtest, only consider an event tradable on `max(MOSTIMPORTANTDATEUTC + ent`

`ry-rule, INGESTDATEUTC)`. Document whichever rule you choose.

8. **No "future" QoQ deltas.** - QoQ features (current quarter minus previous quarter) are fine. "Next quarter minus current" is not.
9. **Corporate-action / delisting handling.** - Don't assume a fill on a non-tradable day. If your price source returns NaN / zero volume / no quote on the would-be entry date, skip the trade or roll forward to the next valid trading day. Document the rule.
10. **Hyperparameter tuning leaks too.**
    - If you grid-search hyperparameters on the full sample and then re-run the backtest with the winners, you have leaked. Tune on a held-out sub-period (e.g., 2010–2017), then freeze and walk-forward from 2018+.

> Print this list, tick every box, and include the signed checklist in your submission.

---

4. Suggested Workflow

You start with one file: the ATC zip. Everything else — price data, universe lists, model code — you build or fetch yourself.

4.1 Suggested project structure

```
project/
├── README.md                  # one-command repro
├── data/
│   ├── load_signals.py        # unzip + load CSV (chunked!) → Parquet cache
│   └── load_prices.py         # fetch and cache daily adj-close prices per ticker
├── features/
│   ├── engineer.py            # AspectTheme aggregations, QoQ, trends
│   └── audit.py               # asserts that prove no look-ahead
├── backtest/
│   ├── splits.py              # walk-forward split generator
│   ├── single_feature_ic.py   # part 1
│   ├── quintile.py            # part 2
│   ├── model.py               # part 3 (model wrapper + walk-forward loop)
│   └── portfolio.py           # part 4 (rebalanced portfolio sim)
├── reports/
│   ├── charts.py              # all matplotlib code
│   └── pdf.py                 # reportlab writer
└── results/                   # outputs (gitignored)
```

4.2 Practical tips

- **Memory.** The CSV is 4.5 GB. Don't `pd.read_csv()` the whole thing. Load in chunks of 100k rows, drop the unused ~580 AspectTheme columns up-front (keep only what your features use), and persist a Parquet cache. After the first load you'll be working with a ~200 MB Parquet file.

- **Drop `delete` rows immediately.** Filter `SignalType != 'delete'` before anything else.
- **Pick one SignalType for the main analysis** (recommend `Total`). Run alternatives only after the `Total` pipeline works.
- **Compute returns once and cache.** Forward-return computation is the slowest non-model step. Save the returns-augmented dataframe to Parquet and reuse it.
- **Join prices on `BESTTICKER`.** That's the cleanest field for matching against an external price feed (yfinance, etc.). Cache prices per ticker once and reuse.
- **Walk-forward = quarterly.** Train end ≤ 2019, test 2020Q1 → 2026Q2 one quarter at a time.
- **5 bps per side for transaction costs.** Justify any deviation.
- **Always print sample sizes per quarter.** A quarter with <100 events is too small to draw quintile conclusions from.
- **Sanity check: market neutrality.** A long-only strategy on US equities will look great in 2010–2021 even with a random signal. Always report the **long-short** spread, not just long-only.

4.3 Common pitfalls students hit

- "My Sharpe is 4." → You leaked. Audit §3 again.
- "My signal works on SP500 but not on RU3K." → Plausible (small caps are noisier). Don't tune your model differently per universe just to get a positive number; that's overfitting to the test set.
- "Daily rebalance crushes monthly." → Check turnover and post-cost Sharpe. Daily L/S strategies often die after costs.
- "I removed sectors that lost money." → Look-ahead. You wouldn't have known which sectors would lose at the time.
- "My model is great in 2020." → 2020 is a regime-change year. If your edge is concentrated in COVID, say so.

---

5. Grading Rubric (40 / 30 / 20 / 10)

Weight	Criterion
40%	**Correctness — no look-ahead bias.** Failing the audit is an automatic ≤50% on this section.
30%	**Quality of analysis.** Were the right experiments run? Are conclusions supported by the data? Is uncertainty quantified?
20%	**Production-readiness of recommendation.** Is your suggested deployment realistic — cadence, capacity, costs, risk?
10%	**Code quality & reproducibility.** One command from raw CSV → final PDF.

---

6. Resources

6.1 The signal data

Dataset (presigned URL, valid for 7 days from 2026-04-22 — i.e. expires 2026-04-29; request a refresh after that):

```
https://pronto-misc.s3.amazonaws.com/Earnings_ATC_until_2026-04-21.csv.zip?AWSAccessKeyId=AKIA
24FT6FZHXN7GZ34Y&Signature=17M0%2FEkltHUJUVC7sa66WF2EmPI%3D&Expires=1777452260
```

Direct download (≈422 MB compressed → ≈4.5 GB CSV). After expiration, ask the instructor to regenerate.

This zip is **the only file you receive for the project**. Everything else — daily prices, universe constituents, transaction-cost assumptions — you source or assume yourself, and document in your write-up.

6.2 Sourcing your own price data

You will need **daily adjusted close prices** for every ticker that appears in the signal file (≈19,800 unique tickers across the full universe; far fewer once you restrict to S&P 500 / S&P 1500 / Russell 3000). Recommended approach:

- `yfinance` (free, Python, keyed by ticker symbol) is sufficient for this project. Use `BESTTICKER` from the signal file as the join key.
- Fetch each ticker once, cache to disk (Parquet or per-ticker CSV), and **never re-hit the API mid-experiment** (it will rate-limit you and the inconsistency wrecks reproducibility).
- Alternatives if you have access: Tiingo, Polygon, Alpha Vantage, Compustat (the academic license at most universities), Bloomberg, FactSet.
- Whichever source you use, **always use *adjusted* close** (split- and dividend-adjusted). Unadjusted prices will produce nonsense returns around corporate actions.
- For Russell 3000, expect 10–20% of tickers to be untradable on a given day (delistings, M&A, suspensions). Handle gracefully — see audit item §3.9.

6.3 Universe membership

Today's S&P 500 is not 2014's S&P 500. Best practice is to use a point-in-time constituents file (Wikipedia historical tables, CRSP, Compustat, or commercial vendors). If you cannot obtain one, document the survivorship-bias caveat explicitly in your research PDF and apply your model on a "current-membership" universe — your reported alpha will be an upper bound.

Good luck. The signal does have alpha — the question is how much survives a clean backtest, and whether you can shape it into something tradeable.